

```

}
strcpy(sigerrmsg, "SIGINT Raised");
printf("sigerrmsg in main before
raise %s\n", sigerrmsg);
raise(SIGINT);
printf("sigerrmsg in main after
raise %s\n", sigerrmsg);
return 0;
}

```

Here's an example of a signal handler with global memory [Char *sigerrmsg] with an atomic object [flag]:

```

//compliantatomi.c
#include<stdio.h>
#include<signal.h>
#include<stdlib.h>
#include<string.h>

#define MSGSIZE 24
volatile sig_atomic_t flag=0;
char *sigerrmsg;
void handler (int signum)
{
    flag = 1; // set is atomic
    operation signal cpu write cycle
}
int main()
{
    signal(SIGINT, handler);
    sigerrmsg=(char *)malloc(MSGSIZE);
    if(!sigerrmsg) {
        printf("Memory not
allocated\n");
        exit(0);
    }
    strcpy(sigerrmsg, "SIGINT Raised");
    printf("sigerrmsg in main before
raise %s\n", sigerrmsg);
    raise(SIGINT);
    if(flag == 1) { // get is atomic
        operation single cpu read cycle
        strcpy(sigerrmsg, "SIGINT
Received");
    }
    printf("sigerrmsg in main after
raise %s\n", sigerrmsg);
    return 0;
}

```

Output of compliant and non-

compliant atomic code is given in Figure 3. Here, the output remains but non-compliant atomic code is subject to undefined behaviour. A good practice of programming in a signal handler when there is a shared object is to use the atomic object volatile `sig_atomic_t` which ensures read/set and write/get operations that complete in a single CPU cycle.

Call-back function using structures and function pointers

A structure creates a data type that can be used to group items of possibly different types into a single type. The function pointer is just a pointer to a C function. Unlike normal pointers, a function pointer points to code, not data. Typically, a function pointer stores the start of executable code; we do not allocate or de-allocate memory using function pointers. The function's name can also be used to get its address. In GCC, by using structure and function pointers, we can create re-usable C++ objects and its member functions.

An example of creating structure variables dynamically and invoking its member functions using function pointers (call-back functions) is given below:

```

//callbackstruct.c
#include<stdio.h>
#include<stdlib.h>
int gStatus = 0;
typedef struct Callback {    //
    Reusable structure
    int (*read)();           //
    Function pointers for call Back
    int (*write)(int Status);
} CB;

int readData()               //
Reusable Function
{
    printf("gstatus value in read
%u\n", gStatus);
    return gStatus;
}

```

```

int writeData(int Status) // Reusable
Function
{
    gStatus = Status;
    printf("gstatus value in write
%u\n", gStatus);
}
CB * CallbackStructCreate()
{
    CB *cb = (CB *)malloc(sizeof(CB));
    if(cb)
    {
        cb->read = &readData; //
        Initialize function pointers
        cb->write = &writeData;
    }
    return cb;
}
void CallbackStructFree(CB *cb)
{
    if(cb)
        free(cb);
}
int main()
{
    CB *cbtest = CallbackStructCreate();
    // Create require instance dynamically
    if (cbtest) {
        cbtest->write(20);
        cbtest->read();
    }
    CallbackStructFree(cbtest);
    return 0;
}

```

Figure 4 gives the output of read/writes with a structure function pointer [callback methods].

There's a lot to explore in GCC, which will enable us to grow as professional programmers. 

 By: K. Chandramohan

The author is senior technical lead at HCL Technologies, Chennai. He has 15 years of experience in networking and embedded product enhancement and sustenance. His interests include Snort and OpenSSL, system programming in C and C++, and data structures.